

LAPORAN RESMI
“PENGENALAN PEMROGRAMAN
BERBASIS OBYEK”



Disusun oleh:

Nama: Kristian Utama Putra

NRP: 3125600047

Kelas: 2 STr. Teknik Informatika B

Politeknik Elektronika Negeri Surabaya

2026

Tugas Pendahuluan

1. Apakah yang dimaksud dengan kelas, method, atribut dan obyek?

- **Class:** blueprint/template untuk membuat objek
- **Method:** fungsi yang dimiliki obyek
- **Atribut:** isi atau variable yang menyimpan data di dalam objek
- **Obyek:** entitas konkret dari sebuah class

Analogi:

- **Class** = mobil
- **Method** = maju, mundur, belok
- **Atribut** = warna, kecepatan
- **Obyek** = mobil saya

2. Buatlah contoh suatu kelas dan definisikan atribut dan methodnya!

Ada class dengan nama “Siswa”,

Mempunyai atribut “nrp” dan “nama”,

Dan mempunyai method bernama yang berfungsi menjelaskan bahwa siswa ini adalah siswa PENS.

3. Buatlah kode program soal no. 2 diatas!

```
public class Siswa {  
    public int nrp;  
    public String nama;  
  
    public void info() {  
        System.out.println("Ini siswa PENS");  
    }  
}
```

4. Buatlah kelas yang berisi main method yang membuat obyek dari kelas yang telah dibuat di soal no. 3. Selanjutnya obyek tersebut mengakses atribut dan methodnya.

```
public class Main {  
    public static void main(String[] args) {  
        Siswa s = new Siswa();  
  
        s.nrp = 2024001;  
        s.nama = "Budi";  
  
        s.info();  
  
        System.out.println("NRP: " + s.nrp);  
        System.out.println("Nama: " + s.nama);  
    }  
}
```

Eksekusi:

```
PS E:\Politeknik Elektronika Negeri Surabaya (PENS)\Semester 3\OOP\W2> javac Siswa.java  
PS E:\Politeknik Elektronika Negeri Surabaya (PENS)\Semester 3\OOP\W2> javac Main.java  
PS E:\Politeknik Elektronika Negeri Surabaya (PENS)\Semester 3\OOP\W2> java Main  
Ini siswa PENS  
NRP: 2024001  
Nama: Budi  
PS E:\Politeknik Elektronika Negeri Surabaya (PENS)\Semester 3\OOP\W2> _
```

Percobaan

Percobaan 1: Mengakses anggota suatu class

```
public class Siswa {  
    int nrp;  
    public void setNrp(int i) {  
        nrp=i;  
    }  
}
```

```
public class Test {  
    public static void main(String args[]) {  
        Siswa anak=new Siswa();  
        anak.setNrp(5);  
        System.out.println(anak.nrp);  
    }  
}
```

```
PS E:\Politeknik Elektronika Negeri Surabaya (PENS)\Semester 3\OOP\W2\Percobaan> javac Test.java  
PS E:\Politeknik Elektronika Negeri Surabaya (PENS)\Semester 3\OOP\W2\Percobaan> javac Siswa.class  
error: invalid flag: Siswa.class  
Usage: javac <options> <source files>  
use --help for a list of possible options  
PS E:\Politeknik Elektronika Negeri Surabaya (PENS)\Semester 3\OOP\W2\Percobaan> javac Test.java  
PS E:\Politeknik Elektronika Negeri Surabaya (PENS)\Semester 3\OOP\W2\Percobaan> java Test  
5
```

Percobaan 2 : Mengakses anggota suatu class

```
public class Siswa {  
    int nrp;  
    String nama;  
  
    public void setNrp(int i) {  
        nrp=i;  
    }  
  
    public void setNama(String i) {  
        nama=i;  
    }  
}
```

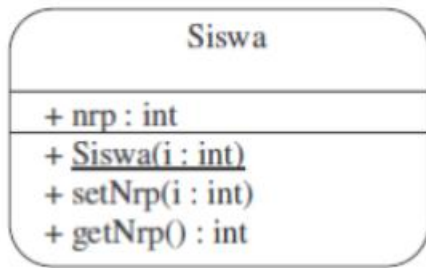
Eksekusi:

```
PS E:\Politeknik Elektronika Negeri Surabaya (PENS)\Semester 3\OOP\W2\Percobaan> javac Test.java  
PS E:\Politeknik Elektronika Negeri Surabaya (PENS)\Semester 3\OOP\W2\Percobaan> java Test  
5
```

Analisis Percobaan 1 dan 2:

Objek anak diinstansiasi dari kelas Siswa, lalu atribut nrp dan nama diisi menggunakan method mutator setNrp() dan setNama() alih-alih diakses langsung, yang menunjukkan awal penerapan prinsip encapsulation meskipun atributnya sendiri masih bersifat public sehingga tetap bisa diakses langsung via anak.nrp; pola ini menegaskan bahwa method dapat menjadi jembatan pengubah nilai objek tanpa mengubah cara akses data secara langsung.

Percobaan 3 : Mengimplementasikan UML class diagram dalam program



```
public class Siswa {
    public int nrp;

    public Siswa(int i) {
        nrp=i;
    }

    public void setNrp(int i) {
        nrp=i;
    }

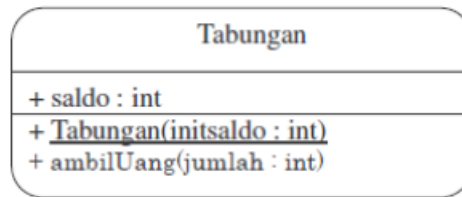
    public int getNrp() {
        return nrp;
    }
}
```

Analisis:

Penerjemahan UML Class Diagram ke kode Java memperlihatkan bahwa tanda + pada diagram konsisten dipetakan menjadi modifier public, sementara penambahan konstruktor Siswa(int i) memungkinkan inisialisasi atribut nrp langsung saat objek dibuat lewat new Siswa(val), dan method getNrp() diperkenalkan sebagai accessor standar untuk membaca nilai atribut tanpa mengaksesnya secara langsung, menandai transisi menuju pola getter yang lebih rapi.

Latihan

Latihan 1 : Mengimplementasikan UML class diagram dalam program untuk class



Transformasikan class diagram diatas ke dalam bentuk program. Tulislah listing program berikut ini sebagai pengetesan.

```
public class TesLatihan1{
    public static void main(String args[]){
        Tabungan tabungan = new Tabungan(5000);
        System.out.println("Saldo awal : " + tabungan.saldo);
        tabungan.ambilUang(2300);
        System.out.println("Jumlah uang yang diambil : 2300");
        System.out.println("Saldo sekarang : " + tabungan.saldo);
    }
}
```

Lakukan kompilasi pada program diatas dan jalankan. Jika tampilan di layar tampak seperti dibawah ini, maka program anda sudah benar. Jika tidak sama, benahi kembali program anda dan lakukan hal yang sama seperti diatas.

```
Saldo awal : 5000
Jumlah uang yang diambil : 2300
Saldo sekarang : 2700
```

```
public class Tabungan {
    public int saldo;

    public Tabungan(int initsaldo) {
        this.saldo = initsaldo;
    }

    public void ambilUang(int jumlah) {
        this.saldo -= jumlah;
    }
}
```

```
public class Tes1 {  
    public static void main(String args[]) {  
        Tabungan tabungan = new Tabungan(5000);  
        System.out.println("Saldo awal : " + tabungan.saldo);  
  
        tabungan.ambilUang(2300);  
        System.out.println("Jumlah uang yang diambil : 2300");  
        System.out.println("Saldo sekarang : " + tabungan.saldo);  
    }  
}
```

Eksekusi:

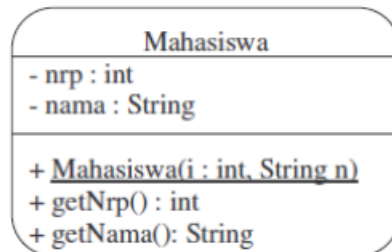
```
PS E:\Politeknik Elektronika Negeri Surabaya (PENS)\Semester 3\OOP\W2\Latihan> java Tes1  
Saldo awal : 5000  
Jumlah uang yang diambil : 2300  
Saldo sekarang : 2700
```

Analisis:

Konstruktor `Tabungan(int initsaldo)` berhasil menginisialisasi saldo awal sebesar 5000 menggunakan `this.saldo`, dan pemanggilan method `ambilUang(2300)` melakukan operasi pengurangan langsung tanpa validasi apa pun, sehingga saldo akhir bernilai 2700 sesuai kalkulasi matematis biasa — latihan ini menjadi dasar sebelum konsep validasi transaksi diperkenalkan pada tugas berikutnya.

Latihan 2 : Mengimplementasikan UML class diagram dalam program untuk class Mahasiswa

Latihan 2 : Mengimplementasikan UML class diagram dalam program untuk class Mahasiswa



Transformasikan class diagram diatas ke dalam bentuk program! Tulislah listing program berikut ini sebagai pengetesan.

```
public class TesLatihan2{
    public static void main(String args[]){
        Mahasiswa mhs = new Mahasiswa(12345, "Jono");
        System.out.println("NRP : " + mhs.getNrp());
        System.out.println("Nama : " + mhs.getnama());
    }
}
```

Lakukan kompilasi pada program diatas dan jalankan. Jika tampilan di layar tampak seperti dibawah ini, maka program anda sudah benar. Jika tidak sama, benahi kembali program anda dan lakukan hal yang sama seperti diatas.

```
NRP : 12345
Nama : Jono
```

```

public class Mahasiswa {
    private int nrp;
    private String nama;

    public Mahasiswa(int i, String n) {
        this.nrp = i;
        this.nama = n;
    }

    public int getNrp() {
        return nrp;
    }

    public String getNama() {
        return nama;
    }
}

```

```

public class Tes2 {
    public static void main(String args[]) {
        Mahasiswa mhs = new Mahasiswa(12345, "Jono");
        System.out.println("NRP : " + mhs.getNrp());
        System.out.println("Nama : " + mhs.getNama());
    }
}

```

Eksekusi:

```

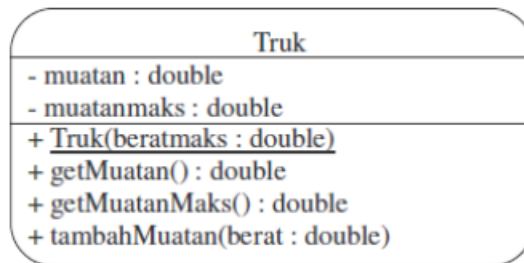
PS E:\Politeknik Elektronika Negeri Surabaya (PENS)\Semester 3\OOP\W2\Latihan> java Tes2
NRP : 12345
Nama : Jono

```

Analisis:

Kelas Mahasiswa mulai menerapkan enkapsulasi penuh dengan mendeklarasikan atribut nrp dan nama sebagai private, sehingga satu-satunya cara mengisi maupun membaca data adalah melalui konstruktor dua-parameter Mahasiswa(int i, String n) untuk inisialisasi sekaligus, dan method getter getNrp()/getNama() untuk pembacaan nilai — pola ini menggambarkan penerapan enkapsulasi yang lebih ketat dibanding Percobaan 1 & 2.

Latihan 3 : Mengimplementasikan UML class diagram dalam program untuk class Truk



Transformasikan class diagram diatas ke dalam bentuk program! Tulislah listing program berikut ini sebagai pengetesan.

```
public class TesLatihan3{
    public static void main(String args[]){
        Truk truk = new Truk(1000);
        System.out.println("Muatan maksimal = "+truk.getMuatanMaks());
        truk.tambahMuatan(500.0);
        System.out.println("Tambah muatan : 500 ");
        truk.tambahMuatan(350.0);
        System.out.println("Tambah muatan : 350 ");
        truk.tambahMuatan(100.0);
        System.out.println("Tambah muatan : 100 ");
        truk.tambahMuatan(150.0);
        System.out.println("Tambah muatan : 150 ");
        System.out.println("Muatan sekarang = " + truk.getMuatan());
    }
}
```

Lakukan kompilasi pada program diatas dan jalankan. Jika tampilan di layar tampak seperti dibawah ini, maka program anda sudah benar. Jika tidak sama, benahi kembali program anda dan lakukan hal yang sama seperti diatas.

```
Muatan maksimal : 1000.0
Tambah muatan : 500
Tambah muatan : 350
Tambah muatan : 100
Tambah muatan : 150
Muatan sekarang = 950.0
```

```

public class Truk {
    private double muatan;
    private double muatanmaks;

    public Truk(double beratmaks) {
        this.muatanmaks = beratmaks;
        this.muatan = 0.0;
    }

    public double getMuatan() {
        return muatan;
    }

    public double getMuatanMaks() {
        return muatanmaks;
    }

    public void tambahMuatan(double berat) {
        if (this.muatan + berat <= this.muatanmaks) {
            this.muatan += berat;
        }
    }
}

```

Eksekusi:

```

PS E:\Politeknik Elektronika Negeri Surabaya (PENS)\Semester 3\OOP\W2\Latihan> java Tes3
Muatan maksimal = 1000.0
Tambah muatan : 500
Tambah muatan : 350
Tambah muatan : 100
Tambah muatan : 150
Muatan sekarang = 950.0

```

Analisis:

Method `tambahMuatan()` dilengkapi logika kondisional `if (muatan + berat <= muatanmaks)` sebagai bentuk validasi bisnis sederhana, dan hal ini terbukti pada penambahan terakhir sebesar 150 yang seharusnya membuat total muatan menjadi 1100 (melebihi batas maksimal 1000), namun karena kondisi bernilai false maka penambahan tersebut diabaikan sistem dan muatan akhir tetap bertahan di 950.0, menunjukkan pentingnya validasi di dalam method untuk menjaga integritas data objek.

Tugas

Tugas 1 : Mengimplementasikan UML class diagram dalam program untuk class Tabungan

```
public class Tabungan {  
    private int saldo;  
  
    public Tabungan(int initsaldo) {  
        this.saldo = initsaldo;  
    }  
  
    public int getSaldo() {  
        return saldo;  
    }  
  
    public void simpanUang(int jumlah) {  
        this.saldo += jumlah;  
    }  
  
    public boolean ambilUang(int jumlah) {  
        if (this.saldo >= jumlah) {  
            this.saldo -= jumlah;  
            return true;  
        } else {  
            return false;  
        }  
    }  
}
```

```
public class Tes1 {  
    public static void main(String srt[]) {  
        boolean status;  
        Tabungan tabungan = new Tabungan(5000);  
        System.out.println("Saldo awal : " +  
tabungan.getSaldo());  
  
        tabungan.simpanUang(3000);  
        System.out.println("Jumlah uang yang disimpan : 3000");  
  
        status = tabungan.ambilUang(6000);  
        System.out.print("Jumlah uang yang diambil : 6000 ");  
        if (status) System.out.println("ok"); else  
System.out.println("gagal");  
  
        tabungan.simpanUang(3500);  
        System.out.println("Jumlah uang yang disimpan : 3500");  
    }  
}
```

```
        status = tabungan.ambilUang(4000);
        System.out.print("Jumlah uang yang diambil : 4000 ");
        if (status) System.out.println("ok"); else
System.out.println("gagal");

        status = tabungan.ambilUang(1600);
        System.out.print("Jumlah uang yang diambil : 1600 ");
        if (status) System.out.println("ok"); else
System.out.println("gagal");

        tabungan.simpanUang(2000);
        System.out.println("Jumlah uang yang disimpan : 2000");
System.out.println("Saldo sekarang = " + tabungan.getSaldo());
    }
}
```

Eksekusi:

```
PS E:\Politeknik Elektronika Negeri Surabaya (PENS)\Semester 3\OOP\W2\Tugas> java Tes1
Saldo awal : 5000
Jumlah uang yang disimpan : 3000
Jumlah uang yang diambil : 6000 ok
Jumlah uang yang disimpan : 3500
Jumlah uang yang diambil : 4000 ok
Jumlah uang yang diambil : 1600 gagal
Jumlah uang yang disimpan : 2000
Saldo sekarang = 3500
```

Analisis:

Method `ambilUang()` dikembangkan agar mengembalikan tipe data boolean sebagai indikator status transaksi — bernilai `true` jika saldo mencukupi dan saldo langsung dikurangi, atau `false` jika tidak; hal ini terbukti saat percobaan menarik 1600 dari saldo tersisa 1500, di mana kondisi `saldo >= jumlah` bernilai `false` sehingga status "gagal" ditampilkan dan saldo tidak berkurang, membuktikan bahwa return value boolean efektif digunakan sebagai mekanisme validasi transaksi yang aman.

Tugas 2 : Menganalisa, membuat UML class diagram dan implementasi program

a. Identifikasi Karakteristik Kolom (Tabel 7.1 Data Karakteristik Mobil):

Obyek	A (Merk)	B (Warna)	C (Tipe)	D (Kapasitas Mesin)	E (Kapasitas Penumpang)
mobil1	Toyota	Biru	Minibus	2000 cc	7 orang
mobil2	Daihatsu	Hitam	Pick up	1500 cc	2 orang
mobil3	Suzuki	Silver	Suv	1800 cc	5 orang
mobil4	honda	merah	sedan	1300 cc	5 orang

b. Desain UML Class Diagram Mobil:

- **Mobil**

- - merk: String

- - warna: String

- - tipe: String

- - kapasitasMesin: int

- - kapasitasPenumpang: int

1300 cc

5 orang

- + Mobil(merk: String, warna: String, tipe: String, kapasitasMesin: int, kapasitasPenumpang: int)

- + infoMobil(): void

```

public class Mobil {
private String merk;
private String warna;
private String tipe;
private int kapasitasMesin;
private int kapasitasPenumpang;

public Mobil(String merk, String warna, String tipe, int kapasitasMesin, int kapasitasPenumpang) {
    this.merk = merk;
    this.warna = warna;
    this.tipe = tipe;
    this.kapasitasMesin = kapasitasMesin;
    this.kapasitasPenumpang = kapasitasPenumpang;
}

public void infoMobil() {
    System.out.println("Merk                : " + merk);
    System.out.println("Warna                : " + warna);
    System.out.println("Tipe                : " + tipe);
    System.out.println("Kapasitas Mesin     : " + kapasitasMesin + " cc");
    System.out.println("Kapasitas Penumpang : " + kapasitasPenumpang + " orang");
    System.out.println("-----");
}
}

```

```

public class TesMobil {
    public static void main(String[] args) {
        Mobil mobil1 = new Mobil("Toyota", "Biru", "minibus", 2000, 7);
        Mobil mobil2 = new Mobil("Daihatsu", "Hitam", "pick up", 1500, 2);
        Mobil mobil3 = new Mobil("Suzuki", "Silver", "suv", 1800, 5);
        Mobil mobil4 = new Mobil("Honda", "Merah", "sedan", 1300, 5);

        System.out.println("=== INFORMASI ARMADA RENTAL MOBIL ===");

        System.out.println("Mobil 1:");
        mobil1.infoMobil();
        System.out.println("Mobil 2:");
        mobil2.infoMobil();
        System.out.println("Mobil 3:");
        mobil3.infoMobil();
        System.out.println("Mobil 4:");
        mobil4.infoMobil();
    }
}

```

Eksekusi:

```

PS E:\Politeknik Elektronika Negeri Surabaya (PENS)\Semester 3\OOP\W2\Tugas> Java Tes2
=== INFORMASI ARMADA RENTAL MOBIL ===
Mobil 1:
Merk           : Toyota
Warna          : Biru
Tipe           : minibus
Kapasitas Mesin : 2000 cc
Kapasitas Penumpang : 7 orang
-----
Mobil 2:
Merk           : Daihatsu
Warna          : Hitam
Tipe           : pick up
Kapasitas Mesin : 1500 cc
Kapasitas Penumpang : 2 orang
-----
Mobil 3:
Merk           : Suzuki
Warna          : Silver
Tipe           : suv
Kapasitas Mesin : 1800 cc
Kapasitas Penumpang : 5 orang
-----
Mobil 4:
Merk           : Honda
Warna          : Merah
Tipe           : sedan
Kapasitas Mesin : 1300 cc
Kapasitas Penumpang : 5 orang

```

Analisis:

Lima karakteristik mobil (merk, warna, tipe, kapasitas mesin, kapasitas penumpang) berhasil dipetakan menjadi atribut private pada kelas Mobil, dengan konstruktor lima-parameter yang memudahkan inisialisasi seluruh data sekaligus saat instansiasi, sementara method infoMobil() menyajikan seluruh informasi secara terformat dan konsisten; empat objek mobil (mobil1–mobil4) yang dibuat dari kelas yang sama ini membuktikan efisiensi dan reusability OOP dalam merepresentasikan banyak entitas serupa dengan struktur data dan perilaku yang seragam.